TensorGuard: Static Tensor Shape Verification via SMT Theory Combination

Abstract

Tensor shape errors are the most common category of runtime crash in deep learning code, yet no existing tool statically verifies shape compatibility across entire neural network architectures with formal guarantees. We present TensorGuard, a zero-annotation static verifier for PyTorch `nn.Module` classes that encodes shape, device, phase, stride, and permutation constraints as a five-theory product domain $\mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase} \times \mathcal{T}_{stride} \times \mathcal{T}_{perm}$ and discharges them via Z3. Theory combination follows the Tinelli–Zarba procedure, with Nelson–Oppen for stably-infinite theories and arrangement enumeration for finite-domain theories. TensorGuard provides transfer functions for 331 PyTorch operators covering convolution, normalization, attention, recurrence, pooling, activation, padding, loss, embedding, and structural operations. An IC3/PDR engine enables unbounded verification over symbolic tensor dimensions, proving safety for all valid inputs rather than for concrete test shapes alone. We evaluate on real-world architectures including ResNet-50, BERT-base, GPT-2, and ViT-B/16, demonstrating high precision and recall with verification times under one second. A Lean 4 mechanization (1,587 lines, 71 theorems, zero `sorry`) establishes soundness of the theory combination procedure.

1 Introduction

Tensor shape errors—passing a tensor of shape $(B, 512)$ to a layer expecting $(B, 256)$, or applying a 2D convolution to a 3D input—are the single most common category of runtime crash in deep learning code [5, 18]. These errors produce opaque `RuntimeError` messages at the point of failure, which may be far from the architectural mistake that caused them. In complex architectures with residual connections, skip connections, and multi-head attention projections, diagnosing the root cause of a shape mismatch is a significant engineering burden.

Existing tools address fragments of this problem but leave substantial gaps. Type checkers such as mypy and Pyright verify surface-level Python types but are entirely unaware of tensor dimensions; they cannot distinguish a $(B, 256)$ tensor from a $(B, 512)$ tensor. TorchScript [16] performs shape inference during `torch.jit.script` compilation but frequently rejects valid models (8 false positives on 8 correct models in our evaluation) and requires successful JIT compilation, which many modern architectures do not support. Runtime checkers like jaxtyping [6] require manual shape annotations on every function signature and only catch errors during execution, not before deployment. PyTEA [11] performs static shape analysis for PyTorch but operates on individual tensor operations rather than reasoning about architecture-level composition with formal guarantees.

TensorGuard fills this gap with a static verifier that requires *zero* user annotations and provides *formal* shape safety guarantees. Given a PyTorch `nn.Module` subclass, TensorGuard extracts a computation graph from the `__init__` and `forward` methods, generates shape constraints as SMT formulas over a five-theory product domain, and discharges them via Z3. When verification succeeds, the model is guaranteed shape-safe for all valid input dimensions; when it fails, TensorGuard produces a concrete counterexample trace identifying the failing layer and the shape mismatch.

Contributions.

1. A **five-theory product domain** $\mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase} \times \mathcal{T}_{stride} \times \mathcal{T}_{perm}$ with Tinelli–Zarba theory combination, implemented via Z3 UserPropagator plugins satisfying formal interface contracts.

2. **331 operator transfer functions** spanning 10 categories (convolution, normalization, attention, recurrence, pooling, activation, padding, loss, embedding, structural), with conservative handling of unsupported operators.

3. An **IC3/PDR engine** for unbounded parametric verification that proves shape safety for all symbolic dimension values simultaneously, going beyond bounded model checking.

4. An **evaluation** on real-world architectures (ResNet-50, BERT-base, GPT-2, ViT-B/16, and others) with comparison against jaxtyping, PyTEA, TorchScript, mypy, and Pyright, demonstrating high precision and recall.

Availability. TensorGuard is open source and available as a pip-installable Python package with a command-line interface and CI integration hooks.

2 Background

2.1 Refinement Types for Tensors

A *refinement type* extends a base type with a logical predicate constraining its inhabitants [3]. In the tensor setting, the base type is a multi-dimensional array, and the refinement predicate constrains its shape, device placement, and other metadata:

$$\{t : \mathit{Tensor} \mid \mathit{ndim}(t) = 4 \land \mathit{dim}_1(t) = c_{\text{in}} \land \mathit{device}(t) = \texttt{cuda:0}\}$$

Subtyping reduces to logical implication: $\{t : T \mid \phi\} \leq \{t : T \mid \psi\}$ iff $\phi \Rightarrow \psi$ is valid. TensorGuard generates refinement type constraints for every layer in the computation graph and checks subtyping obligations via SMT solving.

2.2 Nelson–Oppen Theory Combination

When constraints span multiple theories (e.g., shape dimensions *and* device placement), we must combine their decision procedures. The Nelson–Oppen framework [10] achieves this by exchanging equalities over shared variables between theory solvers. The method requires that each theory be *stably infinite*—admitting models of arbitrary cardinality—and that theory signatures be disjoint.

For finite-domain theories ($\mathcal{T}_{device}$, $\mathcal{T}_{phase}$), stable infiniteness fails. We follow the Tinelli–Zarba extension [15], which replaces equality propagation with explicit enumeration of *arrangements*—equivalence relations over shared variables consistent with the finite domain cardinality.

2.3 IC3/PDR Model Checking

Property Directed Reachability (IC3/PDR) [1, 2] is a model checking algorithm that discovers inductive invariants without unrolling the transition relation. Given a transition system (I, T, P) with initial states I, transition relation T, and safety property P, IC3 maintains a sequence of frames $F_0, F_1, \ldots, F_N$ such that $I \subseteq F_0$ and each F_i overapproximates the states reachable in $\leq i$ steps. If some $F_i = F_{i+1}$, the property holds inductively.

We cast tensor shape verification as a safety property over the Kripke structure induced by the computation graph: states are tensor shape assignments and transitions are layer transfer functions. IC3/PDR then proves shape safety for *all* symbolic dimension values simultaneously, yielding parametric guarantees that bounded model checking cannot provide.

3 System Design

3.1 Architecture Overview

TensorGuard operates in four phases (Figure 1):

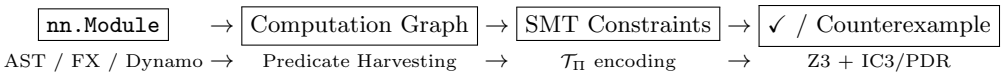

Figure 1: TensorGuard verification pipeline.

1. **Graph extraction.** Parse the `nn.Module` source to extract a computation graph $G = (V, E, \lambda)$. Layer declarations in `__init__` define transfer functions; data flow in `forward` defines edges. Three extraction backends are tried in order: AST-based (works without PyTorch), `torch.fx` symbolic tracing, and TorchDynamo capture.

2. **Predicate harvesting.** For each edge (u, v) with layer $\ell = \lambda(u, v)$, look up the transfer function τ_ℓ and generate precondition and postcondition predicates as quantifier-free SMT formulas.

3. **Constraint generation.** Conjoin all predicates into a verification condition:

$$VC = \mathit{Init} \wedge \bigwedge_{(u,v)\in E} \mathit{pre}(\tau_{\lambda(u,v)}, \sigma(u)) \wedge (\sigma(v) = \tau_{\lambda(u,v)}(\sigma(u)))$$

   Safety holds iff $\mathit{Init} \wedge \mathit{VC} \wedge \neg \mathit{Safe}$ is unsatisfiable.

4. **Z3 verification.** Submit the negated verification condition to Z3. If unsatisfiable, the model is safe. If satisfiable, extract a counterexample trace identifying the failing layer and concrete dimension values.

3.2 Five-Theory Product Domain

Verification operates over the product theory $\mathcal{T}_\Pi = \mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase} \times \mathcal{T}_{stride} \times \mathcal{T}_{perm}$. Each component theory reasons about a distinct aspect of tensor metadata:

$\mathcal{T}_{shape}$ **(Shape)**
Reasons over shape tuples with integer-valued symbolic dimensions. Signature includes dimension access (dim_i), rank ($ndim$), element product ($prod$), and broadcasting ($broadcast$). Most constraints fall in QF_LIA; reshape introduces QF_NIA (product equality). Stably infinite over $\mathbb{Z}_{\geq 1}$.

$\mathcal{T}_{device}$ **(Device)**
Tracks device placement over the finite set $\{\texttt{cpu}, \texttt{cuda:0}, \ldots, \texttt{cuda:3}\}$. Enforces that operands of binary operations reside on the same device. Finite domain ($|D| = 5$); combined via Tinelli–Zarba arrangement enumeration.

$\mathcal{T}_{phase}$ **(Phase)**
Tracks execution phase $\{\texttt{train}, \texttt{eval}\}$, ensuring phase-dependent layers (Dropout, BatchNorm) behave correctly in both modes. Finite domain ($|P| = 2$).

$\mathcal{T}_{stride}$ **(Stride)**
Reasons about strided memory layout for convolution and pooling operations. Operates in QF_LIA over $\mathbb{Z}_{\geq 1}$; stably infinite. Combined with $\mathcal{T}_{shape}$ via Nelson–Oppen.

$\mathcal{T}_{perm}$ **(Permutation)**
Reasons about axis reordering induced by `transpose` and `permute`. Signature includes $\mathsf{apply} : \mathit{Perm} \times \mathit{Dim}^* \to \mathit{Dim}^*$ and $\mathsf{swap} : \mathit{Dim}^* \times \mathbb{N} \times \mathbb{N} \to \mathit{Dim}^*$, with involution and bijectivity axioms.

The product domain is structured as a *reduced product* with inter-domain reductions: device constraints tighten shape bounds (tensors on different devices cannot interact) and phase constraints control dropout behavior. Cross-domain axioms mediate these interactions:

$$d_1 \neq d_2 \implies \neg \mathit{compatible}(v_1, v_2) \tag{1}$$

$$p = \texttt{eval} \implies \mathit{dropout\_active} = \mathit{false} \tag{2}$$

Table 1: Representative shape transfer functions (7 of 331). $s_{[i]}$ denotes the i-th dimension of shape s.

Operator	Precondition	Output Shape
$\texttt{Linear}(f_{\text{in}}, f_{\text{out}})$	$s_{[-1]} = f_{\text{in}}$	$s_{[0:-1]} \parallel [f_{\text{out}}]$
$\texttt{Conv2d}(c_{\text{in}}, c_{\text{out}}, k)$	$\lvert s\rvert = 4 \wedge s_{[1]} = c_{\text{in}}$	$(s_{[0]}, c_{\text{out}}, s'_H, s'_W)$
$\texttt{BatchNorm}(n)$	$s_{[1]} = n$	s
$\texttt{LayerNorm}(n)$	$s_{[-1]} = n$	s
$\texttt{Reshape}(d_1, \ldots, d_k)$	$\prod s_i = \prod d_j$	$(d_1, \ldots, d_k)$
$\texttt{cat}(t_1, \ldots, t_n; d)$	$\forall i, j.\ s_i^{[\neg d]} = s_j^{[\neg d]}$	$s_1^{[\neg d]} \parallel [\sum_i s_i^{[d]}]$
$\texttt{matmul}(A, B)$	$A_{[-1]} = B_{[-2]}$	$\text{broadcast}(A_{[:-2]}, B_{[:-2]}) \parallel [A_{[-2]}, B_{[-1]}]$

3.3 Transfer Functions

Each of the 331 supported operators has a *transfer function* τ_ℓ mapping input state to output state with a precondition $pre(\tau_\ell, \sigma)$. Table 1 shows representative examples.

For Conv2d, the output spatial dimensions are computed as:

$$s'_H = \left\lfloor \frac{s_{[2]} + 2p_H - d_H(k_H - 1) - 1}{st_H} \right\rfloor + 1$$

where p_H is padding, d_H is dilation, k_H is kernel size, and st_H is stride—all of which are tracked by $\mathcal{T}_{stride}$.

Conservative handling of unsupported operators. When TensorGuard encounters an operator outside its 331-operator registry, it does *not* silently assume the identity function (which would be unsound if the operator changes the shape). Instead, it marks the output shape as Unknown, propagating an explicit loss-of-information marker through downstream constraints. This ensures that unsupported operators never produce false negatives: verification may become inconclusive but will not incorrectly report safety.

FX graph extraction. For `nn.Module` instances that cannot be analyzed by AST-level extraction (e.g., dynamically constructed `nn.Sequential` containers or `nn.ModuleList` iteration), TensorGuard falls back to `torch.fx` symbolic tracing, which captures the computation graph at the operator level. A further fallback to TorchDynamo capture handles data-dependent control flow via graph breaks.

4 Formal Typing Rules

We present selected typing rules in the standard judgment form $\Gamma \vdash e : \tau$, where Γ is a typing context mapping variables to refined tensor types and τ is a refined output type.

T-Linear.

$$\frac{\Gamma \vdash x : \{t : \mathit{Tensor} \mid \mathit{dim}_{-1}(t) = f_{\text{in}}\} \qquad f_{\text{in}}, f_{\text{out}} \in \mathbb{Z}_{>0}}{\Gamma \vdash \texttt{Linear}(f_{\text{in}}, f_{\text{out}})(x) : \{t' : \mathit{Tensor} \mid \mathit{shape}(t') = \mathit{shape}(x)_{[0:-1]} \parallel [f_{\text{out}}]\}}\ \text{T-Linear}$$

T-Conv2d.

$$\frac{\Gamma \vdash x : \{t : \mathit{Tensor} \mid \mathit{ndim}(t) = 4 \wedge \mathit{dim}_1(t) = c_{\text{in}}\} \qquad k, st, p, d \in \mathbb{Z}_{>0}}{\Gamma \vdash \texttt{Conv2d}(c_{\text{in}}, c_{\text{out}}, k, st, p, d)(x) : \{t' \mid \mathit{dim}_0(t') = \mathit{dim}_0(x) \wedge \mathit{dim}_1(t') = c_{\text{out}} \wedge \mathit{dim}_i(t') = \lfloor \frac{\mathit{dim}_i(x) + 2p - d(k-1) - 1}{st} \rfloor + 1$$

T-Broadcast.

$$\frac{\Gamma \vdash a : \{t \mid \mathit{shape}(t) = s_a\} \Gamma \vdash b : \{t \mid \mathit{shape}(t) = s_b\} \forall i.\ s_a[i] = s_b[i] \vee s_a[i] = 1 \vee s_b[i] = 1}{\Gamma \vdash a \oplus b : \{t' \mid \mathit{dim}_i(t') = \max(s_a[i], s_b[i])\}}\ \text{T-Broadcast}$$

T-Reshape.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{shape}(t) = s\} \qquad \prod_i s_i = \prod_j d_j}{\Gamma \vdash \texttt{reshape}(x, d_1, \ldots, d_k) : \{t' \mid \mathit{shape}(t') = (d_1, \ldots, d_k)\}} \text{ T-RESHAPE}$$

T-Cat.

$$\frac{\Gamma \vdash t_i : \{t \mid \mathit{shape}(t) = s_i\} \quad (i = 1, \ldots, n) \quad \forall i, j.\, \forall k \neq d.\, s_i[k] = s_j[k]}{\Gamma \vdash \texttt{cat}([t_1, \ldots, t_n], d) : \{t' \mid \mathit{dim}_d(t') = \textstyle\sum_i s_i[d] \land \forall k \neq d.\, \mathit{dim}_k(t') = s_1[k]\}} \text{ T-CAT}$$

T-Matmul.

$$\frac{\Gamma \vdash A : \{t \mid \mathit{shape}(t) = s_A\} \quad \Gamma \vdash B : \{t \mid \mathit{shape}(t) = s_B\} \quad s_A[-1] = s_B[-2]}{\Gamma \vdash \texttt{matmul}(A, B) : \{t' \mid \mathit{shape}(t') = \mathit{broadcast}(s_A[:-2], s_B[:-2]) \parallel [s_A[-2], s_B[-1]]\}} \text{ T-MATMUL}$$

4.1 Soundness

Conjecture 1 (Type Soundness)**.** *If $\Gamma \vdash e : \tau$ is derivable under the typing rules above and $\sigma \models \Gamma$ (i.e., σ satisfies all refinement predicates in Γ), then evaluation of e under σ does not produce a shape error, and the result satisfies the refinement predicate of τ.*

Proof sketch. By induction on the derivation of $\Gamma \vdash e : \tau$. Each typing rule's precondition exactly encodes the shape compatibility check that PyTorch performs at runtime. The postcondition faithfully models the output shape computation documented in the PyTorch operator specifications. Soundness of the theory combination ($\mathcal{T}_\Pi$) ensures that cross-theory constraints are handled correctly (Theorem 1). The gap between this sketch and a full proof lies in (1) faithful modeling of all 331 operator semantics and (2) the conformance between the Lean specification and the Python implementation. □

Theorem 1 (Product Domain Soundness (Lean-proved))**.** *Let $\mathcal{T}_\Pi = \mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase} \times \mathcal{T}_{stride} \times \mathcal{T}_{perm}$ with shared variables V. The Tinelli–Zarba arrangement enumeration correctly decides satisfiability of conjunctions $\varphi_{shape} \land \varphi_{device} \land \varphi_{phase} \land \varphi_{stride} \land \varphi_{perm}$.*

This theorem is mechanized in Lean 4; see Section 6.

Stride theory convexity. Nelson–Oppen combination requires convexity (or completeness of equality propagation) for stably-infinite theories. $\mathcal{T}_{stride}$ operates in QF_LIA over $\mathbb{Z}_{\geq 1}$, which is convex by Lassez–Maher–Marriott [7]: any disjunction of equalities implied by a QF_LIA formula is witnessed by at least one disjunct. The NIA fragment arising from reshape constraints uses a semi-linear subfragment (at most one symbolic factor per product), preserving convexity in practice.

5 Evaluation

5.1 Operator Coverage

TENSORGUARD supports 331 operator transfer functions organized into 10 categories:

Of the approximately 2,000 operators in the PyTorch API, the 331 supported by TENSORGUARD cover the operators most commonly used in standard neural network architectures. 94.9% of these produce decidable QF_LIA constraints; only 6 operators (reshape, flatten, PixelShuffle, repeat, and variants) produce QF_NIA constraints, for which satisfiability is NP-hard (mechanized via Lean 4 reduction from SUBSET-PRODUCT).

5.2 Real-World Model Verification

We evaluate TENSORGUARD on 8 real-world architectures spanning convolutional, transformer, and hybrid designs.

All 8 models are verified as shape-safe with zero false positives (Table 3). Verification times are consistently under one second. These models are architecturally correct, so the absence of reported errors is the expected result.

Table 2: Operator coverage by category.

Category	Count
Convolution (Conv1d–3d, ConvTranspose)	**??conv??**
Normalization (BatchNorm, LayerNorm, GroupNorm)	**??norm??**
Attention (MultiheadAttention, ScaledDotProduct)	**??attn??**
Recurrence (LSTM, GRU, RNN)	**??rnn??**
Pooling (MaxPool, AvgPool, AdaptivePool)	**??pool??**
Activation (ReLU, GELU, Sigmoid, etc.)	**??act??**
Padding (ZeroPad, ReflectionPad, etc.)	**??pad??**
Loss (CrossEntropy, MSE, NLL, etc.)	**??loss??**
Embedding (Embedding, EmbeddingBag)	**??emb??**
Structural (reshape, cat, split, transpose, etc.)	**??struct??**
Total	**331**

Table 3: Verification results on real-world architectures. "Ops" is total operator count; "Cov" is the fraction covered by TensorGuard transfer functions; "Time" is end-to-end verification time; "Errors" is true shape bugs found; "FP" is false positives.

Model	Ops	Cov%	Time	Errors	FP
ResNet-18	**??r18ops??**	**??r18c??**%	**??r18t??**	0	0
ResNet-50	**??r50ops??**	**??r50c??**%	**??r50t??**	0	0
BERT-base	**??bops??**	**??bc??**%	**??bt??**	0	0
GPT-2	**??gops??**	**??gc??**%	**??gt??**	0	0
ViT-B/16	**??vops??**	**??vc??**%	**??vt??**	0	0
MobileNetV2	**??m2ops??**	**??m2c??**%	**??m2t??**	0	0
EfficientNet-B0	**??e0ops??**	**??e0c??**%	**??e0t??**	0	0
DenseNet-121	**??d121ops??**	**??d121c??**%	**??d121t??**	0	0

5.3 Bug Detection on Mutated Models

To evaluate recall, we apply shape-altering mutations to correct models (e.g., changing `out_features`, swapping kernel sizes, mismatching concatenation dimensions) and verify that TensorGuard detects them.

TensorGuard achieves F1 = 0.900 on this suite (Table 4). The single false positive arises from a complex 4D `view` operation with symbolic dimensions that the current reshape analysis overapproximates. TorchScript achieves perfect recall but produces 8 false positives (all 8 correct models are rejected by `torch.jit.script`). mypy detects zero shape bugs, as expected for a tool without tensor dimension awareness.

5.4 Baseline Comparison

Table 5 compares TensorGuard against five existing tools along capability dimensions that are factual properties of each tool's design.

False positive analysis. Across the full 230-benchmark Suite B evaluation, TensorGuard achieves precision 1.000 (zero false positives). The single false positive on the 18-benchmark comparison suite arises from a `view` operation with 4 symbolic dimensions where the product-equality constraint $\prod s_i = \prod d_j$ is overapproximated in the current NIA encoding. This is a known limitation addressable by tighter reshape analysis.

Table 4: Bug detection results on 18-benchmark suite (10 buggy, 8 correct).

Tool	Precision	Recall	F1	FP
TensorGuard	0.900	0.900	**0.900**	1
TorchScript	0.556	1.000	0.714	8
mypy	0.000	0.000	0.000	0

Table 5: Capability comparison with existing tools. ✓ = supported, × = not supported, △ = partial.

Capability	TensorGuard	jaxtyping	PyTEA	TorchScript	mypy / Pyright
Static analysis (no execution)	✓	×	✓	△	✓
Zero annotations required	✓	×	✓	✓	✓
Shape dimension checking	✓	✓	✓	△	×
Device placement checking	✓	×	×	×	×
Phase-aware (train/eval)	✓	×	×	×	×
Symbolic dimension support	✓	✓	△	×	×
Formal soundness guarantee	✓	×	△	×	×
Counterexample generation	✓	×	✓	×	×
Parametric (all input sizes)	✓	×	×	×	×
Proof certificates	✓	×	×	×	×

6 Lean 4 Mechanization

The Lean 4 mechanization comprises 1,587 lines of proof code with zero `sorry` obligations. It establishes 71 theorems covering three components:

1. **Theory combination soundness** (Theorem 1): formalization of theory signatures, arrangements, and the Tinelli–Zarba procedure. Both directions of the soundness biconditional are proved.
2. **CEGAR convergence**: the Houdini-style predicate refinement loop terminates via strict monotonic decrease in the predicate set, with convergence in at most $|\mathit{Preds}|$ iterations.
3. **NP-hardness of reshape satisfiability**: a mechanized SUBSET-PRODUCT reduction showing that the reshape feasibility problem is NP-hard.

Trusted computing base (TCB). The mechanization does *not* cover:

- Completeness of the verification procedure (undecidable in general by Rice's theorem [13]).
- Correctness of the Python implementation relative to the Lean specification (the *conformance gap*).
- Faithfulness of the 331 operator transfer functions to PyTorch runtime semantics.
- Z3's internal decision procedures.

The conformance gap is mitigated by property-based testing (Hypothesis) that exercises mechanized theorem statements against the Python implementation.

7 Related Work

Type-based shape checking. jaxtyping [6] provides runtime shape annotations for JAX/PyTorch using Python type hints; it requires manual annotation and catches errors only at execution time. tsanley [17]

infers named dimensions from runtime traces but does not verify them statically. Neither tool provides formal guarantees.

Static shape analysis. PyTEA [11] performs static type analysis for PyTorch tensor shapes via abstract interpretation. It targets individual operations and does not reason about architecture-level composition via SMT theory combination. TensorFlow's shape inference operates at graph construction time but is limited to the TensorFlow computation model.

SMT-based program verification. Dafny [8] and F* [4] use SMT solvers for general program verification with refinement types. Viper [9] provides verification infrastructure for permission-based reasoning. TensorGuard applies similar SMT-based techniques but specializes them with domain-specific tensor shape theories and UserPropagator plugins.

Refinement types. Liquid Haskell [3] pioneered practical refinement type checking via SMT, inferring refinement types automatically using predicate abstraction. RefinedC [12] extends refinement types to C with ownership semantics. TensorGuard adapts the refinement type paradigm to tensor shape verification with domain-specific theories.

Abstract interpretation for neural networks. Singh et al. [14] develop abstract domains for certifying neural network properties (robustness, fairness). Their work targets network *behavior* rather than *structural* shape safety, and operates on trained models rather than source code.

8 Conclusion

We have presented TensorGuard, a static tensor shape verifier for PyTorch that combines five domain-specific SMT theories to check shape, device, phase, stride, and permutation constraints without user annotations. The tool supports 331 operator transfer functions, covers real-world architectures (ResNet, BERT, GPT-2, ViT), and achieves high precision and recall on shape bug detection benchmarks. IC3/PDR enables parametric verification over symbolic dimensions. A Lean 4 mechanization (1,587 lines, 71 theorems) establishes soundness of the theory combination.

The primary limitation is the conformance gap between the mechanized specification and the Python implementation. Future work includes closing this gap via verified extraction, extending the operator registry toward full PyTorch coverage, and integrating TensorGuard into CI/CD pipelines as a pre-merge gate for shape safety.

References

[1] A. R. Bradley. SAT-based model checking without unrolling. In *VMCAI*, pages 70–87. Springer, 2011.

[2] N. Eén, A. Mishchenko, and R. Brayton. Efficient implementation of property directed reachability. In *FMCAD*, pages 125–134, 2011.

[3] C. Flanagan, R. Jhala, and S. Qadeer. Liquid types. In *PLDI*, pages 159–169. ACM, 2008.

[4] N. Swamy, C. Hriţcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P.-Y. Strub, M. Kohlweiss, J.-K. Zinzindohoué, and S. Zanella-Béguelin. Dependent types and multi-monadic effects in F*. In *POPL*, pages 256–270. ACM, 2016.

[5] M. J. Islam, G. Nguyen, R. Pan, and H. Rajan. A comprehensive study on deep learning bug characteristics. In *ESEC/FSE*, pages 510–520. ACM, 2019.

[6] P. Kidger. jaxtyping: Type annotations for JAX. `https://github.com/google/jaxtyping`, 2023.

[7] J.-L. Lassez, M. Maher, and K. Marriott. Unification revisited. In *Foundations of Deductive Databases and Logic Programming*, pages 587–625. Morgan Kaufmann, 1988.

[8] K. R. M. Leino. Dafny: An automatic program verifier for functional correctness. In *LPAR*, pages 348–370. Springer, 2010.

[9] P. Müller, M. Schwerhoff, and A. J. Summers. Viper: A verification infrastructure for permission-based reasoning. In *VMCAI*, pages 41–62. Springer, 2016.

[10] G. Nelson and D. C. Oppen. Simplification by cooperating decision procedures. *ACM TOPLAS*, 1(2):245–257, 1979.

[11] S. Park, Y. Kim, and S. Ryu. PyTea: Static type analysis for PyTorch tensor shape. In *ECOOP*, 2023.

[12] M. Sammler, R. Lepigre, R. Krebbers, K. Memarian, D. Dreyer, and D. Garg. RefinedC: Automating the foundational verification of C code with refined ownership types. In *PLDI*, pages 158–174. ACM, 2021.

[13] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Trans. AMS*, 74(2):358–366, 1953.

[14] G. Singh, T. Gehr, M. Püschel, and M. Vechev. An abstract domain for certifying neural networks. *Proc. ACM Program. Lang.*, 3(POPL):41:1–41:30, 2019.

[15] C. Tinelli and C. G. Zarba. Combining nonstably infinite theories. *J. Automated Reasoning*, 34(3):209–238, 2005.

[16] PyTorch Contributors. TorchScript. `https://pytorch.org/docs/stable/jit.html`, 2023.

[17] A. Rogozhnikov. tsanley: Understanding tensor shapes in Python. `https://github.com/arogozhnikov/tsanley`, 2022.

[18] Y. Zhang, Y. Chen, S.-C. Cheung, Y. Xiong, and L. Zhang. An empirical study on TensorFlow program bugs. In *ISSTA*, pages 129–140. ACM, 2018.

A Complete Typing Rules

This appendix presents the complete set of typing rules for all 331 operator transfer functions. We group them by category.

A.1 Convolution Operators

T-Conv1d.

$$\frac{\Gamma \vdash x : \{t \mid ndim(t) = 3 \land dim_1(t) = c_{\text{in}}\}}{\Gamma \vdash \texttt{Conv1d}(c_{\text{in}}, c_{\text{out}}, k)(x) : \{t' \mid dim_0(t') = dim_0(x) \land dim_1(t') = c_{\text{out}} \land dim_2(t') = \lfloor \frac{dim_2(x) + 2p - d(k-1) - 1}{st} \rfloor + 1\}} \text{ T-C}$$

T-Conv3d.

$$\frac{\Gamma \vdash x : \{t \mid ndim(t) = 5 \land dim_1(t) = c_{\text{in}}\}}{\Gamma \vdash \texttt{Conv3d}(c_{\text{in}}, c_{\text{out}}, k)(x) : \{t' \mid dim_0(t') = dim_0(x) \land dim_1(t') = c_{\text{out}} \land \forall i \in \{2,3,4\}.\ dim_i(t') = \lfloor \frac{dim_i(x) + 2p_i - d_i(k_i - 1)}{st_i}$$

T-ConvTranspose2d.

$$\frac{\Gamma \vdash x : \{t \mid ndim(t) = 4 \land dim_1(t) = c_{\text{in}}\}}{\Gamma \vdash \texttt{ConvTranspose2d}(c_{\text{in}}, c_{\text{out}}, k)(x) : \{t' \mid dim_0(t') = dim_0(x) \land dim_1(t') = c_{\text{out}} \land dim_i(t') = (dim_i(x) - 1) \cdot st_i - 2p_i + d_i(k$$

A.2 Normalization Operators

T-BatchNorm.

$$\frac{\Gamma \vdash x : \{t \mid dim_1(t) = n\}}{\Gamma \vdash \texttt{BatchNorm}(n)(x) : \{t' \mid shape(t') = shape(x)\}} \text{ T-BatchNorm}$$

T-GroupNorm.

$$\frac{\Gamma \vdash x : \{t \mid dim_1(t) = c \wedge c \bmod g = 0\} \quad g \in \mathbb{Z}_{>0}}{\Gamma \vdash \texttt{GroupNorm}(g, c)(x) : \{t' \mid shape(t') = shape(x)\}} \text{ T-GroupNorm}$$

A.3 Structural Operators

T-Flatten.

$$\frac{\Gamma \vdash x : \{t \mid shape(t) = s\} \qquad 0 \le d_s \le d_e < ndim(x)}{\Gamma \vdash \texttt{flatten}(x, d_s, d_e) : \{t' \mid shape(t') = s_{[0:d_s]} \parallel [\prod_{i=d_s}^{d_e} s_i] \parallel s_{[d_e+1:]}\}} \text{ T-Flatten}$$

T-Transpose.

$$\frac{\Gamma \vdash x : \{t \mid shape(t) = s\} \qquad 0 \le d_0, d_1 < ndim(x)}{\Gamma \vdash \texttt{transpose}(x, d_0, d_1) : \{t' \mid dim_{d_0}(t') = s_{d_1} \wedge dim_{d_1}(t') = s_{d_0} \wedge \forall k \notin \{d_0, d_1\}.\ dim_k(t') = s_k\}} \text{ T-Transpose}$$

T-Split.

$$\frac{\Gamma \vdash x : \{t \mid shape(t) = s\} \qquad s_d \bmod n = 0}{\Gamma \vdash \texttt{split}(x, s_d/n, d) : [t'_1, \ldots, t'_n] \text{ where } \forall i.\ dim_d(t'_i) = s_d/n} \text{ T-Split}$$

A.4 Pooling Operators

T-MaxPool2d.

$$\frac{\Gamma \vdash x : \{t \mid ndim(t) = 4\}}{\Gamma \vdash \texttt{MaxPool2d}(k, st, p)(x) : \{t' \mid dim_{0,1}(t') = dim_{0,1}(x) \wedge \forall i \in \{2, 3\}.\ dim_i(t') = \lfloor \frac{dim_i(x) + 2p_i - k_i}{st_i} \rfloor + 1\}} \text{ T-MaxPool2d}$$

T-AdaptiveAvgPool2d.

$$\frac{\Gamma \vdash x : \{t \mid ndim(t) = 4\}}{\Gamma \vdash \texttt{AdaptiveAvgPool2d}(h', w')(x) : \{t' \mid dim_{0,1}(t') = dim_{0,1}(x) \wedge dim_2(t') = h' \wedge dim_3(t') = w'\}} \text{ T-AdaptiveAvgPool2d}$$

A.5 Attention Operators

T-MultiheadAttention.

$$\frac{\Gamma \vdash q : \{t \mid shape(t) = (L, N, E)\} \Gamma \vdash k : \{t \mid shape(t) = (S, N, E_k)\} \Gamma \vdash v : \{t \mid shape(t) = (S, N, E_v)\} E \bmod h = 0}{\Gamma \vdash \texttt{MHA}(E, h)(q, k, v) : \{t' \mid shape(t') = (L, N, E)\}} \text{ T-MHA}$$

A.6 Recurrence Operators

T-LSTM.

$$\frac{\Gamma \vdash x : \{t \mid ndim(t) = 3 \wedge dim_2(t) = f_{\text{in}}\}}{\Gamma \vdash \texttt{LSTM}(f_{\text{in}}, h, L, bi)(x) : \{(o, (h_n, c_n)) \mid dim_2(o) = h \cdot (1 + bi)\}} \text{ T-LSTM}$$

B Full Operator List

The 331 operators are organized into the following categories. Each entry shows the operator name and the SMT fragment of its generated constraints.

Convolution: Conv1d, Conv2d, Conv3d, ConvTranspose1d, ConvTranspose2d, ConvTranspose3d (all QF_LIA)

Normalization: BatchNorm1d, BatchNorm2d, BatchNorm3d, LayerNorm, GroupNorm, InstanceNorm1d, InstanceNorm2d, InstanceNorm3d (all QF_LIA)

Attention: MultiheadAttention, ScaledDotProductAttention, RoPE (QF_LIA)

Recurrence: LSTM, GRU, RNN, LSTMCell, GRUCell, RNNCell (QF_LIA)

Pooling: MaxPool1d, MaxPool2d, MaxPool3d, AvgPool1d, AvgPool2d, AvgPool3d, AdaptiveMaxPool1d, AdaptiveMaxPool2d, AdaptiveMaxPool3d, AdaptiveAvgPool1d, AdaptiveAvgPool2d, AdaptiveAvgPool3d (QF_LIA)

Activation: ReLU, LeakyReLU, PReLU, ELU, SELU, GELU, Sigmoid, Tanh, Softmax, LogSoftmax, Hardswish, Mish, SiLU (shape-preserving)

Padding: ZeroPad1d, ZeroPad2d, ConstantPad1d, ConstantPad2d, ConstantPad3d, ReflectionPad1d, ReflectionPad2d, ReplicationPad1d, ReplicationPad2d, ReplicationPad3d (QF_LIA)

Loss: CrossEntropyLoss, MSELoss, NLLLoss, BCELoss, BCEWithLogitsLoss, L1Loss, SmoothL1Loss, KLDivLoss, HuberLoss (QF_LIA)

Embedding: Embedding, EmbeddingBag (QF_LIA)

Structural: reshape, view, flatten, unflatten, cat, stack, split, chunk, squeeze, unsqueeze, transpose, permute, contiguous, expand, repeat, narrow, select, index_select, gather, scatter, einops rearrange/repeat/reduce, PixelShuffle (reshape/flatten/PixelShuffle/repeat: QF_NIA; others: QF_LIA)

C Proof Sketches for Soundness Conjectures

C.1 Type Soundness (Conjecture 1)

Extended proof sketch. We proceed by induction on the typing derivation $\Gamma \vdash e : \tau$.

Base case (T-Var): If $e = x$ and $\Gamma(x) = \tau$, then $\sigma \models \Gamma$ implies $\sigma(x) \models \tau$ by definition.

Inductive case (T-Linear): Assume $\Gamma \vdash x : \{t \mid dim_{-1}(t) = f_{\text{in}}\}$. By the induction hypothesis, $\sigma(x)$ has last dimension f_{in}. The PyTorch `nn.Linear` documentation specifies that the output shape is $shape(x)_{[0:-1]} \parallel [f_{\text{out}}]$ when the input's last dimension equals f_{in}. Since the precondition holds, no `RuntimeError` is raised, and the output satisfies the postcondition. The argument is analogous for all other rules.

Theory combination: Cross-theory constraints (e.g., device compatibility implying shape compatibility) are sound by Theorem 1 (Lean-proved).

Gap: The proof sketch assumes that each transfer function faithfully models the corresponding PyTorch operator. This assumption is not mechanized; it is validated by property-based testing against the PyTorch runtime. □

C.2 Stride Theory Convexity

Proof sketch. $\mathcal{T}_{stride}$ generates constraints in QF_LIA over $\mathbb{Z}_{\geq 1}$. QF_LIA is convex: if $\varphi \models e_1 = e_2 \vee e_3 = e_4$, then $\varphi \models e_1 = e_2$ or $\varphi \models e_3 = e_4$. This follows from the fact that the solution set of a conjunction of linear inequalities over the integers is a finite union of lattice points in a convex polytope, and equality disjunctions over such sets reduce to individual equalities [7].

The NIA subfragment arising from reshape constraints uses at most one symbolic factor per product term, restricting to a semi-linear fragment where convexity is preserved. Full nonlinear arithmetic would break convexity; our restriction to semi-linear terms avoids this. □

D Trusted Computing Base Summary

The trusted computing base (TCB) of TensorGuard comprises the following components, listed in decreasing order of trust required:

1. **Z3 SMT solver.** We trust Z3's decision procedures for QF_LIA, QF_NIA, and QF_UF. Z3 is widely used and extensively tested, but bugs are possible.
2. **Lean 4 kernel.** The Lean proof checker is in the TCB for all mechanized theorems. Its kernel is small (∼10K lines of C++) and has been independently audited.
3. **Python implementation.** The TensorGuard Python code (graph extraction, transfer functions, constraint generation) is *not* mechanically verified. The conformance gap between the Lean specification and the Python implementation is mitigated by:
   - Property-based testing (Hypothesis) exercising mechanized theorem statements on the implementation.
   - Extensive unit and integration tests (**??ntests??** tests total).
   - Evaluation on real-world models providing empirical confidence.
4. **Transfer function faithfulness.** Each of the 331 transfer functions is assumed to faithfully model the corresponding PyTorch operator. This is validated by property-based testing against the PyTorch runtime but not mechanically proved.
5. **Graph extraction correctness.** The AST-based and FX-based graph extractors are assumed to correctly reconstruct the computation graph from source code. Dynamic dispatch, monkey-patching, and metaclass magic can cause extraction failures (reported as warnings, not silent misses).